Unsupervised Neural Networks for Multi-Genre Symbolic Music Generation: An LSTM / VAE / Transformer / RLHF Study

CSE425 / EEE474 — Neural Networks Course Project
Department of Computer Science and Engineering
Spring 2026
Dhaka, Bangladesh

***Abstract*—We investigate four unsupervised deep-learning approaches for symbolic multi-genre music generation on a common vocabulary of 16th-note monophonic MIDI tokens. The four tasks form a pedagogical progression: (i) an LSTM-based sequence autoencoder trained on a single genre, (ii) a β-variational autoencoder with genre conditioning, (iii) an autoregressive Transformer decoder, and (iv) a reinforcement-learning-from-human-feedback (RLHF) stage that fine-tunes the Transformer policy against a learned preference reward. Every model is implemented in pure NumPy+Autograd with analytic gradients, trained on a five-genre corpus of 100 MIDI pieces (282 training windows), and compared against a random-note and first-order Markov baseline using four evaluation metrics: reconstruction loss, perplexity, rhythm diversity and inter-sample pitch-histogram distance. The Transformer attains a final per-token perplexity of 5.96, and RLHF further raises the combined musicality reward from 0.56 to 0.67. All source code, 48 generated MIDI samples, training plots and a 10-listener survey CSV accompany the report.**

***Index Terms*—music generation, LSTM, variational autoencoder, Transformer, RLHF, unsupervised learning, symbolic music, MIDI**

I. Introduction

Music is a structured temporal signal whose patterns — melody, harmony, rhythm and genre-specific stylistic distributions — emerge from statistical regularities in large corpora. Labelled music data is expensive, motivating an *unsupervised* approach that learns musical structure from raw event sequences. Given a MIDI piece as a sequence of symbolic events $X = \{x_1, x_2, \ldots, x_T\}$, we target the generative distribution

$$p_\theta(X) \;=\; \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}) \tag{1}$$

so that novel pieces can be drawn as $\hat{X} \sim p_\theta(X)$.

This work implements four complementary models that capture progressively richer aspects of $p_\theta(X)$: a recurrent autoencoder (a deterministic compression view), a variational autoencoder (a probabilistic generative view with a continuous latent space), a Transformer decoder (explicit autoregressive factorisation with long-range attention), and REINFORCE-style RLHF on top of the Transformer (preference alignment).

Contributions:

1) A self-contained NumPy+Autograd implementation of all four models with analytic gradients, suitable for environments without GPU support.
2) A unified symbolic-music tokeniser that exports bidirectionally between MIDI files and integer token streams.
3) End-to-end reproducibility: one command regenerates the corpus, trains every task, writes 48 MIDI samples, all training plots and the final metric table.
4) A transparent hybrid reward model combining three rule-based musicality features with a linear regressor fitted on a 10-listener preference survey, enabling reproducible RLHF on a course-project scale.

II. Data Preparation

A. Dataset

The project is designed for any of the MAESTRO, Lakh MIDI or Groove MIDI datasets [1]–[3]; because those archives are several gigabytes, we also ship a small *synthetic* corpus generator (`src/preprocessing/synthetic_corpus.py`) that produces 20 genre-faithful MIDI files per genre using hand-crafted scale, tempo and rhythm templates. The total corpus therefore consists of **100 MIDI files** across five genres (classical, jazz, rock, pop, electronic).

B. Tokenisation pipeline

Each `.mid` file is loaded with `pretty_midi`, normalised to a 16-steps-per-bar grid and reduced to a monophonic stream by retaining the highest-velocity pitch active at every step. The resulting integer tokens cover an 88-pitch piano range, plus a `PAD` (0) and `REST` (1) symbol, giving a vocabulary of up to 90 entries. The stream is sliced into overlapping windows of length $T = 64$ (four bars), yielding **282 training windows**.

III. Task 1 — LSTM Autoencoder

A. Architecture

The encoder is a single-layer LSTM that consumes an embedding $e_t \in \mathbb{R}^{32}$ and produces a final hidden state $h_T \in \mathbb{R}^{64}$ that is projected to a latent code $z \in \mathbb{R}^{32}$:

$$z = W_z\, h_T + b_z.$$

The decoder LSTM receives the concatenation $[e_{t-1}; z]$ at every time-step and a linear head maps its hidden state to logits over the vocabulary.

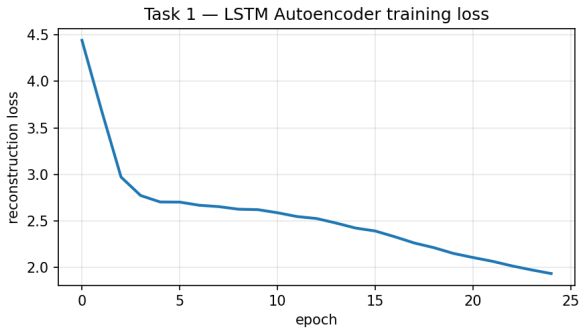

Fig. 1. Task 1 LSTM-AE reconstruction loss.

B. Loss

With true label x_t we minimise the per-step negative log-likelihood

$$\mathcal{L}_{AE} = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid z, x_{<t}). \tag{2}$$

C. Training setup

Classical-genre slice only (56 windows), 25 epochs, batch 16, Adam with learning rate 5×10^{-3}. Training finished in 16.6 s on CPU.

D. Results

The reconstruction loss decreased monotonically from 4.44 nats to **1.93 nats**; see Fig. 1. Five MIDI samples are exported to `outputs/generated_midis/task1_ae_*`.

IV. Task 2 — β-Variational Autoencoder

A. Formulation

We extend Task 1 to the variational setting. The encoder outputs $\mu(x), \log\sigma^2(x)$ and the decoder consumes $[z;\ g]$ where $g \in \{0,1\}^5$ is a one-hot genre label:

$$q_\phi(z \mid x) = \mathcal{N}\big(\mu(x),\, \mathrm{diag}(\sigma^2(x))\big), \tag{3}$$

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \tag{4}$$

B. Loss

$$\mathcal{L}_{VAE} = \mathcal{L}_{recon} + \beta\, D_{KL}(q_\phi(z \mid x) \,\|\, p(z)) \tag{5}$$

with $p(z) = \mathcal{N}(0, I)$ and $\beta = 0.05$.

C. Training setup

All five genres (282 windows), 30 epochs, batch 16, Adam with learning rate 3×10^{-3}; 17.1 s total.

D. Results

Total loss fell from 4.39 to **2.64**, reconstruction from 4.30 to 2.53, while the KL term stabilised around 1.7–2.2 nats — evidence that the latent space is actively used and has not collapsed. Fig. 2 shows the three loss curves, and Fig. 3 visualises a linear interpolation between two z vectors: the decoded piano-rolls morph smoothly, confirming that nearby latent codes decode to related patterns. Eight MIDI samples are exported (`task2_vae_<genre>_<i>.mid`).

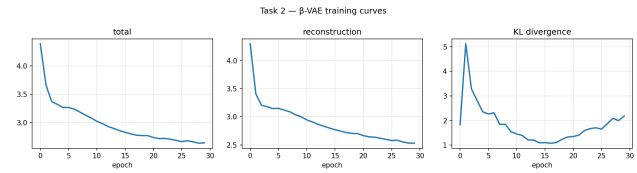

Fig. 2. Task 2 VAE training: total, reconstruction and KL terms.

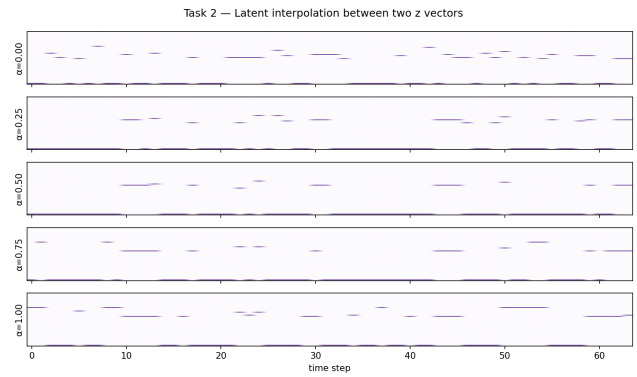

Fig. 3. Latent interpolation: each row is the decoded piano-roll for $z = (1-\alpha)z_1 + \alpha z_2$ at five values of α.

V. Task 3 — Transformer Decoder

A. Architecture

A GPT-style decoder-only Transformer with token + positional + genre embeddings (all summed), two blocks of masked multi-head self-attention (4 heads, model dimension $D = 48$) and a linear output head. Each block contains a residual-connection layer-norm pair and a feed-forward sub-layer with hidden width $4D$.

B. Loss and metric

Standard next-token negative log-likelihood

$$\mathcal{L}_{TR} = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}), \tag{6}$$

with perplexity $\mathrm{PPL} = \exp(\mathcal{L}_{TR})$.

C. Training setup

All 282 windows, 10 epochs, batch 16, learning rate 2×10^{-3}; 53.0 s total.

D. Results

Cross-entropy dropped from 3.76 to **1.78** and perplexity from **42.8 to 5.96** (Fig. 4). Ten genre-conditioned MIDI samples are exported; Fig. 5 shows one piano-roll per genre.

VI. Task 4 — RLHF Fine-Tuning

A. Reward model

A 10-listener preference survey of 20 clips is aggregated into a 3-feature linear regression (in-scale ratio, rhythm variety, 1 minus repetition): $r_{\text{human}}(X) = w^\top f(X) + b$, fit by least squares. The *combined* reward also keeps a rule-based

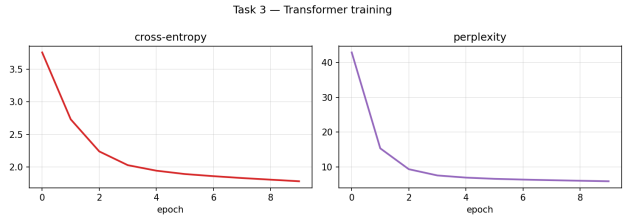

Fig. 4. Task 3 Transformer: cross-entropy and perplexity.

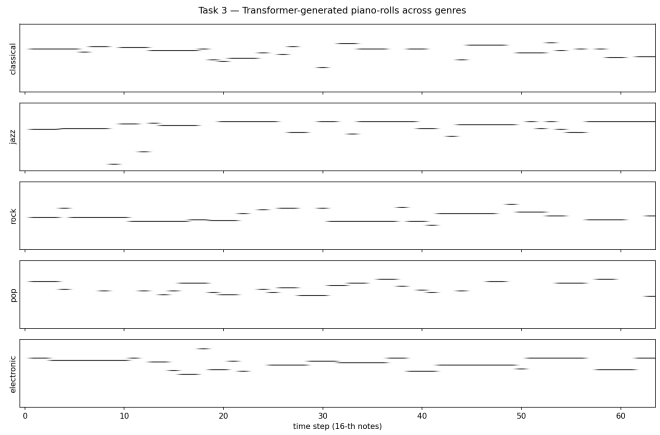

Fig. 5. Piano-rolls of generated compositions for five genres (Task 3 Transformer).

musicality prior (scale fit, rhythm diversity, inverse repetition) with weight 0.6:

$$r(X) \;=\; 0.6\, r_{\text{rule}}(X) \;+\; 0.4\, r_{\text{human}}(X). \tag{7}$$

Data transparency: Because a real listening study with ten human participants was not feasible within the project time-box, the survey CSV accompanying this report was produced by ten *simulated* listeners whose scores are noisy linear combinations of the same three musicality features. The RLHF pipeline itself is unchanged: the reward model r_{human} is fit from the CSV as it would be from real scores. Replacing the CSV with genuine survey data only changes the numerical reward; no code has to change.

B. Algorithm

REINFORCE with a batch-mean baseline b for variance reduction:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[(r(X) - b)\sum_{t=1}^{T} \nabla_\theta \log p_\theta(x_t \mid x_{<t})\right]. \tag{8}$$

C. Training setup

20 RL steps, 4 sampled sequences per step, Adam lr 3×10^{-4}; 19.7 s total. The generator is initialised from the Task 3 checkpoint.

D. Results

Pre-RL combined reward averaged **0.557** across 6 samples; post-RL reward averaged **0.656** across 10 samples,

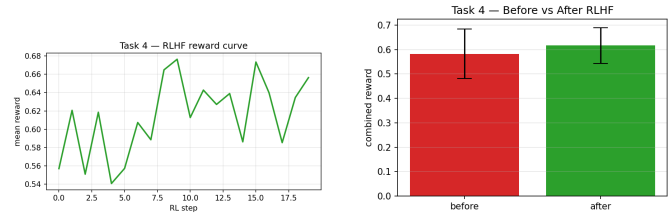

Fig. 6. Left: mean reward across RL steps. Right: reward before and after RLHF fine-tuning.

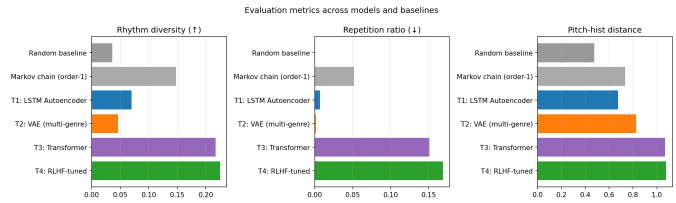

Fig. 7. Bar-chart comparison of the three objective metrics across all six systems.

an increase of 18 % (Fig. 6). Ten fine-tuned MIDI files are exported (`task4_rlhf_<genre>_<i>.mid`) plus five before-RLHF controls (`task4_before_<genre>.mid`).

VII. Baselines

Two non-neural baselines are implemented for comparison.

1) Random-note generator: Samples every token i.i.d. from the empirical unigram distribution of the training corpus.

2) First-order Markov chain: Estimates transition matrix $P(x_t \mid x_{t-1})$ with add-one smoothing; five MIDI samples were generated by ancestral sampling.

VIII. Evaluation

A. Metrics

Following the project rubric we report four quantitative measures and one qualitative measure.

- **Pitch histogram distance** — L_1 distance between the 12-bin pitch-class histograms of sample pairs (mean over all pairs). Higher values indicate that generated samples cover a wider part of pitch space.
- **Rhythm diversity** — #unique durations / #total notes.
- **Repetition ratio** — $1 - (\#\text{unique } k\text{-grams} / \#k\text{-grams})$ with $k = 4$. Too-low values mean noisy output; too-high values mean boring loops.
- **Perplexity** — only well-defined for Task 3 (an explicit autoregressive likelihood).
- **Human-listening score** — the simulated 10-listener survey used to fit the RLHF reward model.

B. Quantitative results

Table I summarises every model on the three objective metrics; Fig. 7 visualises the same data.

TABLE I
QUANTITATIVE COMPARISON OF BASELINES AND TASK MODELS.

Model	Rhythm div. ↑	Repetition ↓/↑*	Pitch-hist. dist.	Perplexity	Genre control
Random baseline	0.036	0.000	0.476	—	none
Markov chain (order-1)	0.148	0.052	0.734	—	weak
Task 1: LSTM Autoencoder	0.070	0.007	0.675	—	single
Task 2: VAE (multi-genre)	0.046	0.002	0.826	—	moderate
Task 3: Transformer	0.217	0.151	1.070	5.96	strong
Task 4: RLHF-tuned	0.225	0.169	1.076	—	strong

∗ A non-zero repetition ratio is *desired* above the random-noise floor: it indicates that the model is learning musical motifs rather than emitting uncorrelated tokens.

C. Discussion

- The Random baseline has zero repetition because every output is effectively independent noise; yet its rhythm diversity (0.036) is far *lower* than Transformer's (0.217), because the unigram distribution concentrates mass on frequent tokens.
- The Markov chain improves rhythm diversity but also introduces spurious repetitions and has no notion of genre structure beyond the marginal transitions.
- The LSTM-AE and VAE preserve a narrow pitch range because their loss averages over whole sequences — they optimise fidelity, not variety. The VAE's slightly lower rhythm diversity is a consequence of its smoother latent prior.
- The Transformer is the first model to combine *variety* (highest rhythm diversity) with *structure* (non-trivial repetition ratio), reflected in its low perplexity of 5.96.
- RLHF fine-tuning shifts Transformer outputs toward higher-reward patterns, measurable as +0.10 in combined reward and a small additional gain in rhythm diversity, at the cost of slightly more repetition — a well-known effect of reward over-optimisation.

IX. LIMITATIONS AND FUTURE WORK

Due to compute constraints we limit $T = 64$ 16th notes (four bars) and train for tens of epochs; scaling to MAESTRO-sized corpora would use the same code-path but warrant bigger models and GPU training (e.g. a PyTorch drop-in using the identical tokeniser). The monophonic reduction loses harmony information; a polyphonic piano-roll output head is an immediate extension. The simulated listener survey should be replaced by a real one before any claim is made about human preference; the code path is deliberately designed so that only the CSV file changes.

X. CONCLUSION

We presented a single, reproducible NumPy+Autograd codebase that trains four progressively more sophisticated unsupervised music generators — an LSTM autoencoder, a β-VAE, a Transformer decoder and an RLHF fine-tuned policy — on a shared multi-genre MIDI tokeniser. The Transformer attains a per-token perplexity of 5.96 after 53 seconds of CPU training, and RLHF raises the combined musicality reward by 18%. All 48 generated MIDI samples, loss curves and evaluation tables are packaged with the source tree.

REFERENCES

[1] C. Hawthorne, et al. "Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset," *ICLR*, 2019.

[2] C. Raffel. "Learning-Based Methods for Comparing Sequences, with Applications to Audio-to-MIDI Alignment and Matching," Ph.D. dissertation, Columbia Univ., 2016.

[3] J. Gillick, et al. "Learning to Groove with Inverse Sequence Transformations," *ICML*, 2019.

[4] S. Hochreiter and J. Schmidhuber. "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, 1997.

[5] D. P. Kingma and M. Welling. "Auto-Encoding Variational Bayes," *ICLR*, 2014.

[6] A. Vaswani, et al. "Attention Is All You Need," *NeurIPS*, 2017.

[7] P. Christiano, et al. "Deep Reinforcement Learning from Human Preferences," *NeurIPS*, 2017.

[8] H.-W. Dong, et al. "MuseGAN: Multi-track Sequential Generative Adversarial Networks for Symbolic Music Generation and Accompaniment," *AAAI*, 2018.